

Project A1: Advanced OS and CPU Feature Exploration

1 System Configuration

1.1 Hardware and Software Environment

All experiments were run on a laptop with an Intel 12th Generation Core i7-1260P processor. The CPU has 12 physical cores and 16 logical threads. Simultaneous multithreading (SMT) is enabled on the performance cores. The system runs 64-bit Linux with the configuration below:

- CPU: Intel Core i7-1260P
- Architecture: x86_64
- Kernel: Linux 6.8.0-83-generic
- Compiler: GCC (system default)
- Cgroups: cgroup v2

All benchmarks were compiled with `-O2`. This gives reasonable optimizations like a typical build, but avoids extreme compiler changes that could hide the performance effects we want to measure.

1.2 Control of Experimental Variability

To make sure performance changes came from OS features (and not random system noise), several controls were used.

Each process was pinned to specific logical CPUs using `taskset`. This stops the Linux scheduler from moving a task between cores while it runs and makes CPU placement predictable. Before testing, the CPU frequency governor was checked to avoid unexpected frequency changes. Tests were run while the system was otherwise idle to reduce variation from frequency scaling and temperature (thermal) effects.

No extra user applications were running during tests. Network-heavy and CPU-heavy background tasks were avoided, and benchmarks were launched from a dedicated terminal session to reduce interference from unrelated processes.

For CPU bandwidth control tests, processes were placed inside systemd-managed control groups using transient scopes. This allowed CPU usage limits to be applied precisely without changing system-wide settings.

Overall, these controls reduced run-to-run variation and helped ensure the results reflect the scheduling, isolation, and memory features being tested.

2 Feature 1: Scheduling and Isolation via CPU Affinity

2.1 Objective

This experiment tests how Linux scheduling and CPU affinity affect runtime for CPU-bound programs. The main focus is how manually pinning tasks changes where they run, how isolated they are, and how that impacts performance compared to the default scheduler.

2.2 Method

A CPU-bound benchmark was used. It repeatedly performs integer operations in a tight loop and measures its own runtime using a monotonic clock. The program reports total runtime in nanoseconds.

Two identical benchmark instances were started at the same time under three setups:

1. **Unpinned / Unpinned:** Both processes ran without any CPU affinity.
2. **Core 2 / Core 2:** Both processes were pinned to the same logical CPU using `taskset`.
3. **Core 2 / Core 4:** Each process was pinned to a different physical core.

Each setup was repeated multiple times to check consistency and average behavior.

2.3 Results

Table 1 shows the average runtime for each setup.

Configuration	Mean Runtime (s)	Relative Slowdown
Unpinned / Unpinned	1.83	1.0×
Core 2 / Core 4	1.85	1.0×
Core 2 / Core 2	3.58	1.95×

Table 1: Execution time comparison under different CPU affinity configurations

When both processes were unpinned, the Linux scheduler usually placed them on different cores, so they could run in parallel. When both were pinned to the same core, runtime almost doubled because they had to take turns (time-share). Pinning them to different cores brought back near-parallel execution and matched the unpinned baseline.

2.4 Discussion

These results show that CPU affinity does not automatically make programs faster. Instead, it gives control over where a task runs. If used poorly (like pinning two CPU-heavy tasks to the same core), it can hurt performance by removing parallelism.

The unpinned case also shows that the default Linux scheduler does a good job spreading CPU-bound work when the system is not busy. In practice, CPU affinity is most useful when you want isolation and predictable placement, not maximum speed.

This experiment shows the trade-off between letting the scheduler manage tasks automatically versus using manual placement for control.

3 Feature 2: SMT (Hyper-Threading) Interference

3.1 Objective

This experiment measures how simultaneous multithreading (SMT) affects performance. On this system, logical CPUs 2 and 3 are SMT siblings, meaning they share one physical core. We compare runtime when two CPU-bound tasks run on SMT siblings versus on different physical cores.

3.2 Method

The same CPU-bound benchmark from Feature 1 was run in pairs. Each run launched two identical instances under three setups:

1. **Unpinned / Unpinned:** Both tasks ran without pinning.
2. **Core 2 / Core 3:** Tasks were pinned to SMT sibling CPUs on the same physical core.
3. **Core 2 / Core 4:** Tasks were pinned to different physical cores.

Each setup was repeated nine times, and runtime was recorded for both tasks.

3.3 Results

Table 2 shows the mean runtime across all runs.

Configuration	Mean Runtime (s)	Relative Slowdown
Unpinned / Unpinned	1.81	1.00×
Core 2 / Core 4	1.80	1.00×
Core 2 / Core 3 (SMT siblings)	1.91	1.06×

Table 2: Effect of SMT on execution time

3.4 Discussion

When tasks ran on different physical cores, they achieved near-parallel performance. When tasks ran on SMT siblings, runtime increased by about 6–8%. Even though both tasks ran at the same time, they competed for shared core resources, so each one made slightly less progress per unit time.

This shows a common SMT trade-off: SMT can improve total core usage, but for CPU-heavy workloads it can reduce performance per thread because hardware resources are shared.

4 Feature 3: Scheduling & Isolation: Control groups (cgroups)

4.1 Objective

This experiment shows how Linux control groups (cgroups) can limit CPU usage and provide scheduling isolation. We measure how execution time changes when a CPU-bound program is restricted to a fixed fraction of CPU time.

4.2 Method

The same CPU-bound benchmark was used again. All runs were pinned to CPU core 2 to remove placement differences.

Two setups were tested:

1. **Baseline:** the process ran normally with no CPU limit.
2. **50% CPU quota:** the process ran inside a systemd-managed cgroup with a 50% CPU quota.

The quota was applied using:

```
sudo systemd-run --scope -p CPUQuota=50% taskset -c 2 ./spin
```

This creates a temporary cgroup and tells the scheduler the process can only run for half of each scheduling period.

Each setup was repeated multiple times and runtime was recorded.

4.3 Results

Table 3 summarizes the runtimes.

Configuration	Mean Runtime (s)	Relative Slowdown
Core 2 (no quota)	1.75	1.0×
50% CPU quota	3.92	2.24×

Table 3: Impact of CPU quota enforced by control groups

4.4 Discussion

With a 50% CPU quota, runtime more than doubled. This makes sense because the scheduler throttles the process after it uses its allowed CPU time. The program still does the same amount of work, but it gets fewer CPU cycles per second.

This also shows the difference between affinity and cgroups. CPU affinity controls *where* a process runs. Cgroups control *how much* CPU time it is allowed to use. Because of that, cgroups give a stronger form of scheduling isolation.

5 Feature 4: Transparent Huge Pages (THP) and TLB Behavior

5.1 Objective

This experiment tests how Transparent Huge Pages (THP) affects address translation. THP can use larger memory pages, which can reduce pressure on the TLB and reduce translation overhead. We measure this using dTLB load misses and CPU cycles.

5.2 Method

A memory benchmark (`mem`) was used with a 512 MB allocation:

```
./mem 512
```

THP was changed using:

```
echo never | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
echo always | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
```

For each THP mode, performance counters were collected using:

```
perf stat -e dTLB-load-misses,cycles ./mem 512
```

Because this CPU is a hybrid design, `perf` sometimes reports counters under `cpu_atom/*` and sometimes under `cpu_core/*`. If both appeared, the values were added to get one total.

5.3 Results

Table 4 shows the mean and standard deviation for each mode.

THP setting	Runs	dTLB-load-misses	Cycles
never	10	714.8 $\pm$ 336.6	1.504M $\pm$ 0.623M
always	5	568.0 $\pm$ 72.5	1.279M $\pm$ 0.157M

Table 4: THP impact on TLB misses and CPU cycles for `./mem 512`

With THP enabled (`always`), dTLB load misses dropped by about 20.5%, and cycles dropped by about 15.0% compared to THP disabled (`never`). This matches the expected benefit of huge pages: fewer translations and fewer TLB misses.

5.4 Discussion

This experiment suggests THP can reduce address-translation cost for memory-heavy work. The total runtime was very small (milliseconds), so wall-clock time can be noisy due to OS overhead and scheduling. Because of that, the hardware counters (TLB misses and cycles) are better signals for this feature.

In short, THP improved translation efficiency on this system by lowering dTLB misses and reducing total cycles for the same memory-access workload.

A Experimental Data

A.1 Feature 1: CPU Affinity

A.1.1 Unpinned / Unpinned

Spin A (ns)	Spin B (ns)
1749533314	1757284059
1723632832	1728191013
1767135445	1770352166
1866780039	1869684303
1880245615	1883014814
1894069409	1894981155
1796985127	1798325434
1802788299	1813016988
1830633625	1840194149

Table 5: Unpinned execution times

A.1.2 Core 2 / Core 2

Spin A (ns)	Spin B (ns)
3460018183	3464365839
3456030213	3454967469
3466347281	3471879793
3457989188	3466240808
3468760844	3474907709
3528999205	3534488507
3618064250	3623915091
3684013649	3686358204
3716096621	3720897477

Table 6: Execution times with both tasks pinned to Core 2

A.2 Shared Data: Core 2 / Core 4 (Used in Features 1 and 2)

Spin A (ns)	Spin B (ns)
1749175653	1752856999
1762641099	1762816062
1756484791	1758163939
1756108273	1761610562
1782974735	1788131365
1822733329	1823546637
1837734318	1841031458
1850593296	1856965497
1876699630	1880341071

Table 7: Execution times with tasks pinned to different physical cores

A.3 Feature 2: SMT Interference

A.3.1 Core 2 / Core 3 (SMT Siblings)

Spin A (ns)	Spin B (ns)
1870713988	1872625488
1874918554	1880074723
1880510118	1886421663
1891764803	1899287343
1901797523	1902367030
1911426122	1916261144
1923665366	1927472668
1934826694	1939995749
1948798370	1951707903

Table 8: Execution times for SMT sibling contention

A.4 Feature 3: Control Groups (CPU Quota)

50% CPU Quota (ns)	Core 2 Baseline (ns)
3952371650	1717857106
3930469402	1718722975
3902769127	1718342356
3907185201	1720268268
3940767235	1729412949
3887441170	1742528704
3921464609	1756688266
3922266055	1794943643
3961443061	1811541988

Table 9: Execution times with and without CPU quota enforcement

A.5 Feature 4: Transparent Huge Pages

A.5.1 THP Disabled (never)

dTLB-load-misses and cycles are shown. If both `cpu_atom` and `cpu_core` counters were reported, they were added together.

dTLB-load-misses	Cycles
611	1309246
581	1256376
614	1259339
576	1204797
583	1108260
1307	2940960
1364	2361473
591	1147363
578	1304060
343	1152164

Table 10: TLB behavior with THP disabled

A.5.2 THP Enabled (always)

dTLB-load-misses	Cycles
610	1331518
614	1154103
576	1205904
599	1174781
441	1530869

Table 11: TLB behavior with THP enabled